Basics of Programming 2 - Final Project

3D Graphics Engine in C++

Nguyen Ba Phi

S3VYH3

Course ID: ENVIAUAA01

Lab group ID: PE08D

Vaitkus Márton

May 24, 2026

Introduction

The objective of this Final Project is the development of a volumetric rendering and simulated light transport engine written in C++. This project addresses the complex problem domain of simulating a physical volumetric display, wherein discrete light projectors emit intersecting rays into a scattering medium to construct 3D objects through additive light intersection. The solution necessitates the application of theoretical concepts including the physics of light absorption, exponential mist transparency falloff, and isotropic scattering.

To achieve this, the project implements mathematical models for 3D ray marching (utilizing a Digital Differential Analyzer algorithm), discrete voxel grids, and energy conservation mechanisms to prevent infinite feedback loops during secondary light bounces. The program also demonstrates core software engineering principles, including the object-oriented paradigm, dynamic memory management, exception handling, and file management.

Terminology and Definitions:

- **Voxel:** A volumetric pixel representing a value on a regular grid in three-dimensional space. In this engine, voxels store color, transparency, and brightness values.
- **Volumetric Rendering:** A technique used to display a 2D projection of a 3D discretely sampled data set (typically a 3D scalar field).
- **Ray Marching:** A rendering method where imaginary rays of light are mathematically "stepped" or "marched" through a volume to determine intersections and light absorption.
- **DDA (Digital Differential Analyzer):** An algorithm used for interpolating variables over an interval between start and end points; used here for efficient 3D ray traversal.
- **Isotropic Scattering:** The process where light hits a particle and scatters evenly in all directions.
- **Ping-Pong Buffer:** A programming technique using two buffers where one is read from and the other is written to, swapping their roles in each pass to prevent data overwriting during calculations like global illumination.

Program Interface

The program operates via a command-line interface (CLI).

Activating the Programming Environment and Execution: The code is designed to be compiled and run on macOS using the native Clang compiler. To compile the program, the user must navigate to the directory containing the source files (user_GUI.cpp and animation.cpp) and execute the following command in the terminal to invoke the g++ compiler with C++17 standards:

```
g++ -std=c++17 user_GUI.cpp animation.cpp -o engine
```

Once compiled successfully, the program is executed by entering the following command:

```
./engine
```

There are no additional parameters required alongside the execution command.

Terminating the Program: The user communicates with the program through a numerical menu system. To terminate the program, the user must enter the number corresponding to the "Exit" option (Option 6) at the main menu prompt. The program will then safely deallocate memory and return to the command line.

Program Execution

The program is a command-line application that operates via a numerical menu system. From an end-user's perspective, the execution revolves around interacting with this menu to generate, manipulate, and visualize 3D objects and volumetric light simulations.

The Main Menu: Upon launching the executable, the user is presented with the main menu and an initially empty 2D render screen. The user navigates the program by entering the integer corresponding to their desired action:

1. **Choose Line Construction:** (Currently reserved for future limitless line drawing implementations).
2. **Create Generic Object:** Allows the user to select from a predefined list of mathematical, parametric shapes (e.g., spheres, cubes, donuts) and render them as either hollow outlines or solid objects.
3. **Ray Tracing Simulation:** Opens a sub-menu dedicated to physical light transport simulations.
4. **Transform Object:** Allows the user to apply 3D rotational matrices to the currently loaded object over a specified duration, animating the rotation frame-by-frame on the screen.
5. **Save/Load Object:** Allows the user to save their current 3D space to a text file for future use, or load a previously saved configuration.
6. **Exit:** Safely terminates the application.

Executing a Simulation (Example Path):

If the user selects "Ray Tracing Simulation," they are presented with a sub-menu to choose between "Static Random Rays" or "Animated Ray Traced Sphere." If the user selects the animated sphere, the program execution flows as follows:

1. The user inputs the dimensions of the 3D space, the mist transparency (which dictates light falloff), and the desired sphere diameter.
2. The user selects a geometric layout for the light projectors (e.g., Tetrahedron or Cube) and defines how many rays each projector should fire.
3. Finally, the user inputs the translation vectors (X, Y, Z movements) and the duration of the animation.

4. The program then executes the simulation, clearing the terminal and rapidly printing the 2D ASCII renders frame-by-frame as the projectors dynamically track and construct the glowing sphere as it moves through the specified coordinates.

Input and Output

User Input Formats: The program primarily relies on standard console input (std::cin) formatted as integers or floating-point numbers.

- **Menu Navigation:** Single integers (e.g., 1, 2, 3).
- **Environmental Parameters:** Dimensions are input as positive integers (e.g., 30 for a 30x30x30 space). Transparency and brightness values are input as floating-point numbers constrained between 0.0 and 1.0 (e.g., 0.95).
- **Transformations:** Rotations and translations expect space-separated values (e.g., entering 10 0 -5 for X, Y, and Z translation).

Screen Output Format: The primary visual output of the program is the "Rendered 2D Front View". The program projects the 3D voxel data onto a 2D grid and maps the cumulative brightness to an ASCII character set (.:-=+*%#@), where a space ( ) represents darkness and the at-symbol (@) represents maximum brightness. This grid is printed directly to the terminal line-by-line.

File Input and Output (Object Presets): When saving or loading objects (Menu Option 5), the program interfaces with .txt files located in the "Object Presets" directory. The format of these files is strictly defined:

- **Header:** The file must begin with the string CUBE followed by the integer dimension N.
- **Voxel Data:** Each subsequent line represents a modified voxel, formatted as space-separated values: X Y Z Transparency Brightness R G B. (e.g., 15 15 15 0.95 120.5 255 255 255).
- **Line Data Header:** Following the voxel data, the string LINES followed by the total count of parametric lines.
- **Line Data:** Each line is formatted as StartX StartY StartZ EndX EndY EndZ Brightness

Program Structure

Data Structures: The foundational unit is the Pixel struct which stores RGB color channels, transparency, and accumulated brightness. These pixels are dynamically allocated into a 3-dimensional vector grid encapsulated by the Cube struct. For physical light transport, the engine utilizes a Vector3 struct for 3D directional math alongside a Ray struct that pairs an origin Point with a Vector3 direction and an intensity value. The Projector struct represents the physical hardware node storing its position, base target, and a collection of emitted rays. Finally, the ConstructedSpace struct binds the 3D Cube and any parametric line data together ensuring proper memory deallocation via its custom destructor.

Core Modules and Algorithms: The program is divided into several logical modules. The Volumetric Physics module contains the shoot_ray function which implements a 3D Digital Differential Analyzer

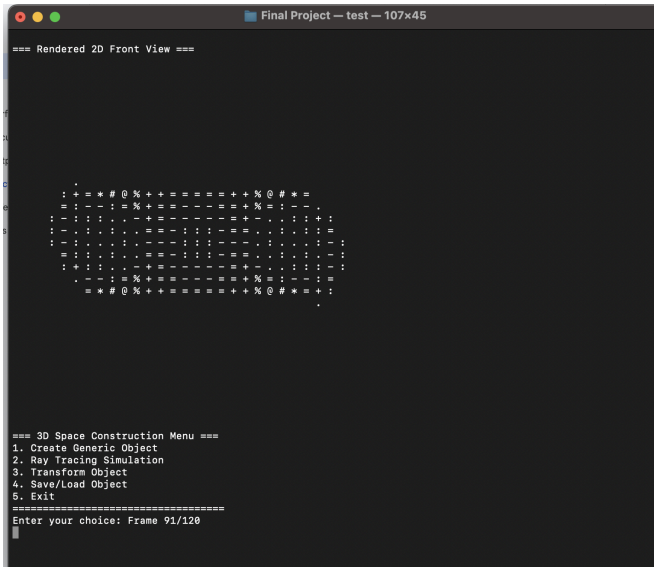
algorithm to step rays through the grid and calculate exponential transparency falloff. It also contains the apply_secondary_lighting function which uses a ping-pong buffer approach for breadth-first global illumination. The Parametric Construction module contains mathematical generators for shapes like spheres and toruses utilizing a 3D Bresenham line algorithm. The Transformation module handles rotational matrices applying backward mapping to prevent rendering artifacts. The Rendering module contains the render_2d_face function which flattens the 3D grid into a 2D ASCII screen by accumulating voxel brightness along the Z-axis. Lastly, the File Management module handles the serialization and deserialization of the voxel space into text documents.

Testing and Verification

Generic object creation:



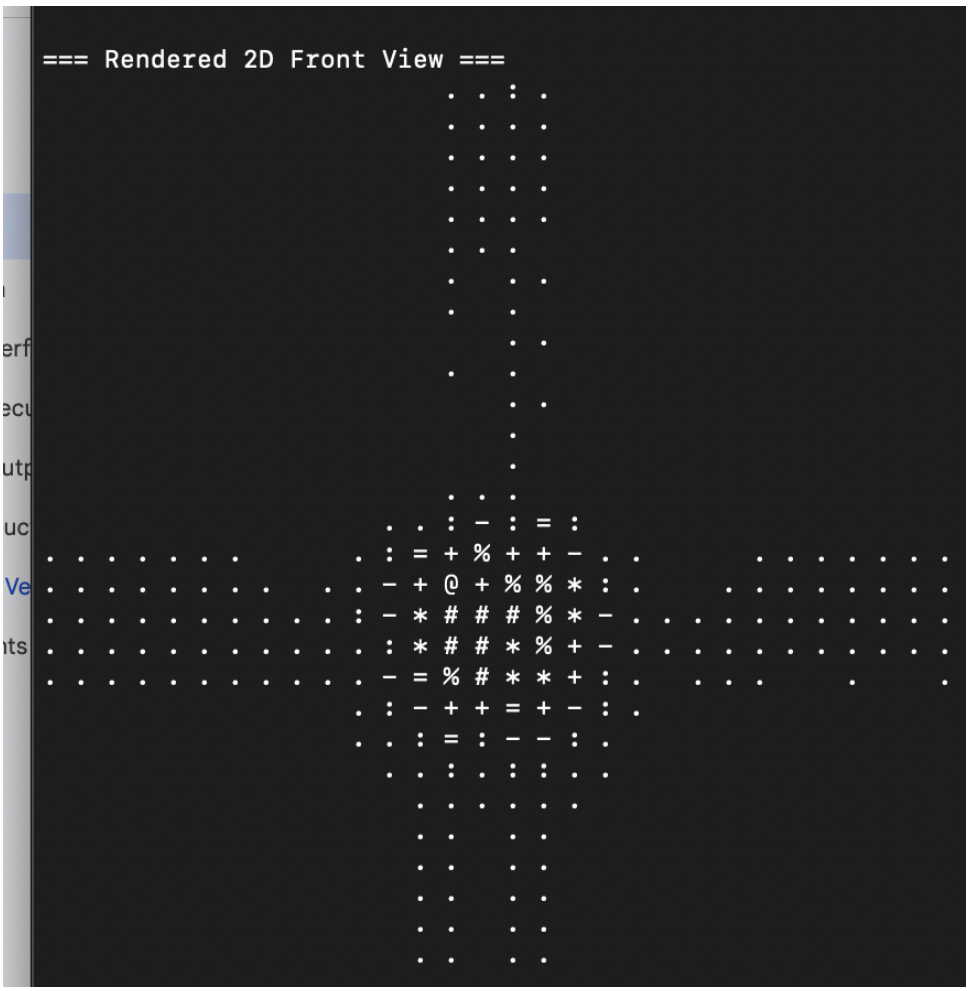
Object rotation



Ray tracing random:



Ray tracing creating a sphere:



Improvements and Extensions

While the current engine successfully demonstrates the fundamental physics of volumetric light transport and object creation, there are several areas for future improvement and extension. One of the primary limitations is the resolution constraint of the ASCII terminal rendering which restricts the visual fidelity of the generated objects.

Future development will focus on expanding the parametric library to support more complex geometric shapes and compound objects. Additionally, the animation system can be extended to support more complex movements including non-linear translation paths, scaling, and dynamic camera perspectives.

The most significant planned extension is the integration of Machine Learning. Currently, the ray-traced objects rely on mathematically generated intersections constrained by predefined bounds. The ultimate goal is to solve the inverse rendering problem by training a neural network. This network will take a desired, complex 3D shape as an input and output the exact matrix of ray angles and intensities required

from the discrete projectors to physically construct that shape in the scattering medium. This will transition the software from a simple renderer into a true control engine for real-world volumetric display hardware.

Conclusion

The development of this volumetric rendering engine successfully demonstrates the intersection of advanced 3D mathematics, physics simulations, and robust software engineering. By building the system entirely from scratch in C++, the project achieved its primary goal of simulating physical light transport through a scattering medium without relying on external libraries or graphics APIs. The implementation of the 3D Digital Differential Analyzer for ray marching and the ping-pong buffer for energy-conserving secondary illumination proved that complex graphical operations can be modeled efficiently using standard C++ data structures.

Furthermore, the project strictly adhered to all course requirements. The object-oriented architecture ensured high modularity, while dynamic memory management was meticulously handled using pointers and custom destructors to prevent memory leaks during frame-by-frame animations. The inclusion of file management for saving and loading volumetric environments, alongside robust exception handling for user inputs, resulted in a highly stable and user-friendly command-line application.

Ultimately, this project provided invaluable experience in translating theoretical physics and geometric concepts into functional software. The challenges overcome during development, particularly regarding memory allocation during rendering loops and backward mapping for 3D rotations, significantly strengthened my understanding of system architecture. The engine now stands as a highly extensible foundation, perfectly positioned for future integration with machine learning algorithms and real-world volumetric display hardware.